A Privacy-Preserving Delegable Auditing Scheme with Edge-Assisted Deduplication in IoT

Chunfei Pan[1], Lei Zhou[2(✉)], Longxia Huang[3], Jiajia Chen[2], Kang Zhang[2], and Anmin Fu[1]

[1] School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing 210094, China
{chunfei.pan,fuam}@njust.edu.cn
[2] School of Information Engineering, Suqian University, Suqian 223800, China
{zhoulei,chenjiajia,zhang_kang}@squ.edu.cn
[3] School of Computer Science and Communication Engineering, Jiangsu University, Zhenjiang 212013, China

Abstract. The rapid growth of Internet of Things (IoT)-generated data poses significant challenges in storage efficiency and integrity verification, particularly for resource-constrained devices. Existing cloud deduplication and auditing schemes incur excessive bandwidth and computational overhead, while failing to ensure privacy and delegability in IoT environments. To address these limitations, we propose a privacy-preserving delegable auditing scheme with edge-assisted deduplication, enabling IoT devices to offload computationally intensive tasks such as authenticator generation and audit interactions to fog nodes via securely negotiated delegable keys. Our approach integrates Message-Locked Encryption for data privacy and homomorphic hash-based block tags that simultaneously serve as deduplication identifiers and integrity proofs, eliminating redundant storage and communication. Unlike conventional cloud-based deduplication, our edge deduplication strategy filters out redundant data before upload, significantly reducing bandwidth consumption. To ensure accountability, we leverage blockchain to manage anonymous task delegation and incentive distribution, preventing impersonation attacks. Additionally, a sampling-based verification mechanism empowers devices to validate fog nodes' honesty by auditing historical logs atomically. Security analysis demonstrates robustness against forgery, while experiments confirm practical efficiency of our proposal.

Keywords: Internet of Things · Cloud storage · Edge deduplication · Integrity auditing · Data privacy

1 Introduction

As a core component of information technology, the IoT has attracted extensive attention from various industries [1]. In a typical IoT system, a large amount of

Y. Xiang and J. Shen (Eds.): CSS 2025, LNCS 16195, pp. 152–167, 2026.
https://doi.org/10.1007/978-981-95-3551-4_11

data is generated by end devices, including RFID tags, sensors, laser scanners, etc. However, processing and analyzing this huge amount of data poses a huge challenge for IoT devices with limited resources [2]. These devices continuously generate data and upload it to support decision making, optimize business processes, and enhance user experience. Nevertheless, as the number of connected devices increases, the amount of data grows exponentially and contains a large amount of duplicate information. The duplicated data not only increase the storage cost, but also reduce the efficiency of data processing and affect the accuracy of analysis results.

With the boom in cloud computing, new possibilities have opened up to meet the challenges of data processing. IoT devices are capable of uploading massive amounts of data to the cloud for analysis, thus effectively alleviating the burden of local computation and maintenance. In this process, data needs to be preprocessed at edge nodes, including filtering and deduplication, to save storage resources [2]. In existing deduplication solutions, the deduplication operation is often performed in the cloud, but transmitting a large amount of data that may contain duplicates to the cloud puts additional pressure on the already strained edge-to-cloud bandwidth, and even if the deduplicated data is processed in the cloud, transmitting the data that should not have been uploaded over long distances to the cloud wastes bandwidth. Consequently, the data uploaded by IoT devices must be deduplicated at the edge node to identify and exclude duplicate information, retain only unique data for subsequent analysis, and then transmit the deduplicated data to the cloud storage server for final storage and analysis. Of course, the Cloud Storage Server (CSS) needs to ensure the integrity of the stored data because outsourcing the data to the cloud implies the transfer of data management to CSS [3]. Although cloud storage servers often claim that the infrastructure they provide is more robust and reliable compared to local facilities, incidents of data loss or corruption still occur from time to time. As the original data copy is removed from the local storage, any damage to the cloud data may result in the uploaded data becoming unusable or even permanently lost. Therefore, ensuring the integrity of cloud stored data has become one of the hot issues [4–6]. Therefore, in IoT scenarios, realizing data edge deduplication and ensuring the integrity of uploaded data are the core requirements for auditing schemes to be applied in IoT environments.

The vast majority of the current auditing schemes cannot be directly applied to IoT data due to the fact that the deduplication requirement has not been realized, while some of the subsequent proposed auditing schemes supporting deduplication are not applicable in IoT environments [7–10]. One of the reasons is that data should be deduplicated at the edge rather than in the cloud to save bandwidth. Another key reason is that IoT devices are mostly designed for data collection and cannot take on the challenge of generating authenticatable tags for the non-stop generated data and launching integrity auditing challenges to the cloud server. Ideally, devices can choose to delegate the time-consuming process of data tagging and auditing interactions to a third-party entity without exposing the raw data. This solution not only reduces the burden on the device

side, but also achieves data integrity protection and security. However, designing an efficient delegatable auditing scheme that supports edge deduplication and privacy preservation in IoT scenarios is a pressing issue.

We propose a delegatable auditing scheme that supports data edge deduplication and privacy protection in an IoT environment. With this scheme, IoT devices are able to delegate complex tag computation and auditing tasks to fog nodes, thereby avoiding incurring heavy computation and communication costs. The tagging computation is inspired by the literature [11] and is improved on it by combining block partitioning techniques. In order to realize data privacy protection, the device utilizes the MLE key to quickly encrypt the original data. The specific contributions can be summarized as follows:

(1) We propose an edge-assisted scheme using identity-based signatures, MLE, and homomorphic hashing to offload tag generation and auditing to fog nodes. Our block-partitioned tagging reduces costs while achieving anonymity protects participant identities.
(2) We implement edge deduplication through MLE-based encryption and block-level processing, where fog nodes perform duplicate detection before cloud transmission. Our unified tag design serves simultaneously as both deduplication identifiers and integrity proofs, eliminating the need for separate authentication metadata in cloud storage.
(3) We develop a blockchain-anchored incentive mechanism with atomic verification, where devices deposit rewards in smart contracts that are only released to fog nodes after successful completion of sampled historical log audits.
(4) We verify the correctness of our scheme and give detailed security proof and evaluate our proposal performance with other schemes by several experiments. The results show that our scheme is effective.

2 Related Works

Ensuring the integrity of cloud data in cloud storage has always been a core research focus. Since Ateniese et al. [6] first proposed Provable Data Possession (PDP) in 2007, which verifies cloud data integrity by generating proofs of possession through randomly sampling data blocks from the server without retrieving all data, this approach has marked significant progress. However, PDP is only applicable to static archived data. Building upon this foundation, researchers have continued to explore the field of data auditing, designing a series of innovative schemes [12–17]. Li et al. [13] simplified the model by retaining only two entities—the data owner and the cloud server, thereby streamlining the auditing process. However, this approach struggles to prevent collusion among malicious cloud servers to forge audit results. Tian et al. [16] introduced a group administrator to detect illegal data, achieving shared auditing with support for distributed trust management. However, existing auditing schemes cannot be directly applied to IoT data because IoT systems generate massive amounts of

redundant data. Directly outsourcing such data for storage would lead to significant resource waste and impose heavy bandwidth pressure on the cloud. To address this issue, many schemes have introduced deduplication-enabled auditing [18–21]. Tian et al. [9] designed a secure deduplication and shared auditing framework featuring a lightweight authenticator and a dual-server model that eliminates the need for a third-party auditor. Although work [21] reduces key management costs by introducing identity-based broadcast encryption, it incurs additional overhead when transmitting metadata for encrypted keys.

However, most existing schemes cannot be effectively applied in IoT environments due to three fundamental limitations. First, the cloud-centric deduplication architecture inevitably requires local devices to transmit both duplicate data blocks and their corresponding authenticators to the cloud for verification, resulting in unnecessary bandwidth consumption between edge and cloud. Second, the computational overhead remains unbalanced, as current solutions demand resource-constrained end devices to generate complex data tags, which overlooks the inherent computational limitations of IoT devices. Furthermore, the lack of privacy preservation mechanisms and incentive structures further restricts practical applicability. These critical gaps highlight the need for an edge-oriented approach that simultaneously addresses efficiency, scalability, and security concerns in IoT deployments.

3 Preliminaries

3.1 Bilinear Maps

Let $\mathbb{G}_1$ and $\mathbb{G}_2$ be multiplicative cyclic groups of prime order q. We define a symmetric bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_1 \to \mathbb{G}_2$ with the following properties:

(1) *Bilinearity*: For all generators $g, h \in \mathbb{G}_1$ and scalars $a, b \in \mathbb{Z}_q$, $e(g^a, h^b) = e(g, h)^{ab}$.
(2) *Non-degeneracy*: For any generator $g \in \mathbb{G}_1$, $e(g, g) \neq 1_{\mathbb{G}_2}$, where $1_{\mathbb{G}_2}$ is the identity element in $\mathbb{G}_2$.
(3) *Computability*: The map e can be evaluated efficiently by a polynomial-time algorithm.

3.2 Hard Problems

Definition 1 (Discrete Logarithm (DL) Problem). *Let $\mathbb{G}_1$ be a cyclic group of prime order q with generator g. For an unknown scalar $a \in \mathbb{Z}_q$, given the pair $(g, g^a) \in \mathbb{G}_1^2$, the DL problem requires computing a. The DL Assumption holds in $\mathbb{G}_1$ if no probabilistic polynomial-time (PPT) algorithm can solve the DL problem with non-negligible advantage.*

Definition 2 (Computational Diffie-Hellman (CDH) Problem). *Let $\mathbb{G}_1$ be a cyclic group as above. For unknown scalars $a, b \in \mathbb{Z}_q$, given the tuple $(g, g^a, g^b) \in \mathbb{G}_1^3$, the CDH problem requires computing $g^{ab} \in \mathbb{G}_1$. The CDH Assumption holds in $\mathbb{G}_1$ if no PPT algorithm can solve the CDH problem with non-negligible advantage.*

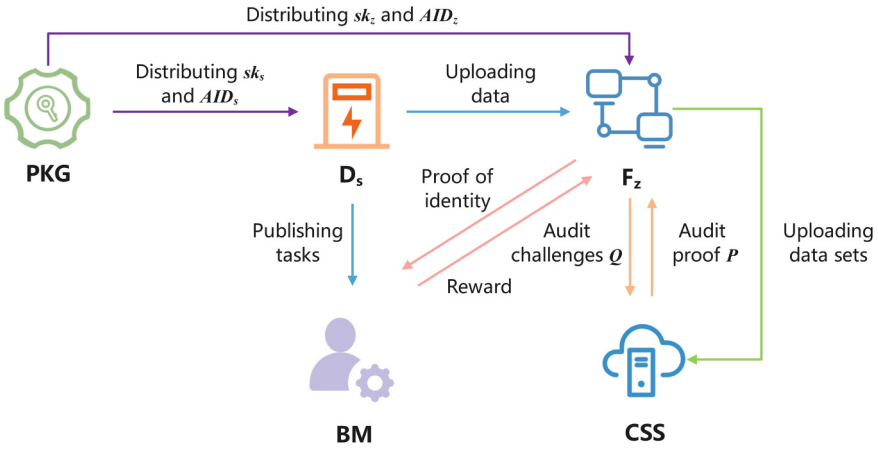


Fig. 1. The system model.

4 System Model and Security Model

4.1 System Model

The present invention relates to a delegatable auditing model supporting data edge de-duplication and privacy protection, which mainly contains the following five entities, as shown in Fig. 1.

(1) IoT Device (D_s): D_s uploads encrypted, chunked data to cloud via fog nodes after deduplication and delegates integrity audits to fog nodes.
(2) Cloud Storage Server (CSS): Resource center with strong compute and storage capabilities. CSS stores data and responds to fog nodes' integrity challenges.
(3) Fog node (F_z): F_z mediates between D_s and CSS, performs data preprocessing and conducts integrity audits for remuneration.
(4) Private Key Generator (PKG): Trusted third party that generates system parameters and issues identity-based keys during initialization.
(5) Blockchain Manager (BM): BM initializes blockchain network comprising IoT devices, CSS and fog nodes, and represents the entire blockchain system.

The interaction process among the five entities is as follows: 1) PKG generates system parameters and distributes private keys and anonymous identities via secure channels, while BM initializes blockchain. D_s, CSS, and F_z register accounts with deposits. 2) D_s publishes tasks; interested F_z negotiates to obtain delegated keys. 3) D_s sends encrypted data to F_z, which deduplicates and uploads to CSS for storage with block tags. 4) During auditing, F_z challenges CSS, verifies integrity proofs, and records results on-chain. 5) Upon D_s's confirmation of valid audit and F_z's identity, blockchain releases task-specific rewards.

4.2 Security Model

We make the following basic assumptions: 1) CSS is semi-honest, properly executing protocols but potentially forging integrity certificates to hide data issues; 2) F_z faithfully performs delegated auditing and deduplication; 3) PKG securely distributes keys and anonymous identities during initialization.

Formally, our scheme achieves the key security objectives, namely 1) F_z can generate block tags, thus preventing CSS and malicious F_z from forging proofs without correct data; (2) Anonymization protects D_s and F_z identities from exposure during audits or blockchain analysis.

The above security can be strictly defined by the adversarial game with two types of adversaries $\mathcal{A}$:

- *External adversary* $\mathcal{A}_1$: $\mathcal{A}_1$ can impersonate a legitimate D_s or F_z and attempt to replace public keys or tamper with delegation protocol parameters;
- *Internal colluding adversary* $\mathcal{A}_2$: $\mathcal{A}_2$ simulates a compromised F_z or PKG, possesses the system master key or delegation key, and attempts to forge block tags or pass audit verification.

The games between $\mathcal{A}$ and a challenger $\mathcal{C}$ will be detailed in the security analysis.

4.3 Design Objectives

The proposed scheme is designed to achieve the following objectives:

(1) *Delegable Auditing*: D_s delegates CSS auditing to F_z, eliminating local block tag generation and reducing D_s's overhead.
(2) *Edge Deduplication*: F_z compares file and block tags to prevent duplicate uploads, saving cloud bandwidth and storage.
(3) *Data Privacy*: MLE encryption with plaintext derived keys ensures only D_s can decrypt intercepted data.
(4) *Identity Privacy*: Anonymization prevents exposure of D_s/F_z identities during transmission/auditing.
(5) *Incentive Mechanism*: Blockchain automatically rewards F_z upon verified audit completion, ensuring fairness.
(6) *Anti-Impersonation*: Identity-verified log audits prevent fraudulent remuneration claims, protecting D_s.

5 Construction

In this section, We propose a privacy-preserving delegable auditing scheme with edge-assisted deduplication IoT and present the detailed of its constructions. Specifically, the scheme involves the following five interaction processes:

5.1 System Initialization

PKG generates necessary parameters. D_s and F_z interact with PKG to obtain their private keys and anonymous identities. Meanwhile, BM initializes the blockchain system, and D_s, CSS, and F_z register blockchain accounts with pre-deposited funds to support subsequent transactions.

- PKG selects two multiplicative cyclic groups G_1 and G_2 of order q and a bilinear pairing $e : G_1 \times G_1 \rightarrow G_2$, where q is a large prime. Four cryptography hash functions are chosen: $H_1 : \{0,1\}^* \times G_1 \rightarrow \mathbb{Z}_q^*$, $H_2 : \mathbb{Z}_q^* \rightarrow \mathbb{Z}_q^*$, $H_3 : \{0,1\}^* \rightarrow \{0,1\}^{256}$, and $H : \{0,1\}^* \rightarrow G_1$. Here, H is a homomorphic hash function, and H_3 is used for MLE key generation. PKG then selects $x \in \mathbb{Z}_q^*$ as the system private key and computes the system public key $X = g^x$. Finally, PKG publishes the system parameters $params = \{G_1, G_2, q, e, H_1, H_2, H_3, H, X\}$, while keeping x secret.
- D_s sends its identity ID_s to PKG. Upon receiving ID_s, PKG randomly selects $r_s \in \mathbb{Z}_q^*$, computes $R_s = g^{r_s}$, and calculates the anonymous identity AID_s and the corresponding private key sk_s as follows:

$$\begin{cases} AID_s = r_s + xH_1(ID_s, R_s) \mod q, \\ sk_s = xH_2(AID_s) \mod q. \end{cases} \tag{1}$$

  PKG sends $\{AID_s, sk_s\}$ to D_s via a secure channel. D_s verifies the consistency of AID_s and sk_s by checking $g^{sk_s} \stackrel{?}{=} X^{H_2(AID_s)}$. If verified, D_s accepts $\{AID_s, sk_s\}$; otherwise, D_s re-requests PKG.
- Similarly, F_z sends its identity ID_z to PKG. PKG randomly selects $r_z \in \mathbb{Z}_q^*$, computes $R_z = g^{r_z}$, and calculates AID_z and sk_z.

$$\begin{cases} AID_z = r_z + xH_1(ID_z, R_z) \mod q, \\ sk_z = xH_2(AID_z) \mod q. \end{cases} \tag{2}$$

  F_z verifies the consistency between AID_z and sk_z using $g^{sk_z} \stackrel{?}{=} X^{H_2(AID_z)}$. If verified, F_z accepts $\{AID_z, sk_z\}$; otherwise, it re-requests.
- BM initializes the blockchain system. D_s, CSS, and F_z register blockchain accounts and pre-deposit funds for supporting subsequent transactions.

5.2 Delegation Key Generation

D_s publishes a task announcement. F_z with idle resources negotiates with D_s to obtain the delegation key for auditing.

- D_s creates and publishes its anonymous identity AID_s and a task announcement $\omega \in \{0,1\}^*$ related to its raw file M. The announcement may include file size, deadline, and reward. If F_z has idle resources and expects to accept tasks, it interacts with D_s to negotiate the delegation key $sk_{s,z}$.

– D_s randomly selects $\nu_{s,z} \in \mathbb{Z}_q^*$, computes $V_{s,z}$ and $sk_{s,z}$ as follows:

$$V_{s,z} = g^{\nu_{s,z}}, sk_{s,z} = \nu_{s,z} + sk_s H_1(\omega, V_{s,z}) \mod q. \tag{3}$$

D_s sends the delegation information $DI = \{\omega, V_{s,z}, sk_{s,z}\}$ to F_z.
– F_z verifies the validity of the delegation information DI by checking $g^{sk_{s,z}} \stackrel{?}{=} V_{s,z} \cdot \left(X^{H_2(AID_s)}\right)^{H_1(\omega, V_{s,z})}$. If invalid, F_z rejects and notifies D_s. Otherwise, F_z accepts DI, saves $sk_{s,z}$, and sends AID_z to D_s.
– D_s uploads $\{\omega, AID_s, AID_z\}$ to the blockchain for storage.

5.3 Data Upload

D_s generates MLE keys to preprocess data, sends it to F_z for deduplication and block tag generation. Then, F_z uploads processed data to CSS.

– D_s splits raw data file M into blocks $\{m_i\}_{1 \le i \le n}$, partitions each block into η segments $m_i = (m_{i1}, m_{i2}, \ldots, m_{i\eta})$, where $m_{ij} \in \mathbb{Z}_q^*$. Each block has η sectors with the same size $\eta|q| = \frac{|M|}{n}$. To protect data privacy, D_s generates an MLE key $k_{ij} = H_3(m_{ij})$ based on plaintext and encrypts the plaintext into ciphertext $C = \{c_{ij}\} = \text{AES.Enc}(k_{ij}, m_{ij})$ using k_{ij}. AES.Enc() refers to the AES-256 algorithm. The purpose of this processing is that no one can generate the corresponding encryption key except for the device that owns the original plaintext. Therefore, any malicious attacker cannot infer plaintext from ciphertext, ensuring that data privacy will not be leaked. Eventually, D_s computes $H(C)$ and sends ciphertext set C and $H(C)$ to F_z.
– F_z checks the consistency between C and $H(C)$. If they are not consistent, it rejects the processing and notifies D_s. Otherwise, further processing is carried out by comparing $H(C)$ with the hash value with the locally stored file. If there are duplicates, the processing request is ignored and D_s is notified that the file is duplicated. Otherwise, F_z chooses two random coefficients $\lambda, \gamma \in \mathbb{Z}_q^*$, a random variable $\varpi \in \mathbb{Z}_q^*$, and η random variables $\varsigma_1, \ldots, \varsigma_\eta$. It computes $a_i = \lambda^i \gamma^i \varpi$ and block tags as follows:

$$\sigma_i = (H(c_{i1}||\cdots||c_{i\eta}) \cdot H(\sum_{j=1}^{\eta} \varsigma_j c_{ij} + a_i))^{sk_{s,z}}. \tag{4}$$

Notably, while we partition each data block into sub-blocks, we compute only one identifier per original block (rather than per sector). This design reduces the number of required block tags to $1/\eta$ compared to conventional schemes, significantly saving cloud storage space. Meanwhile, F_z generates a file tag $T = H(C)^{sk_z}$ and uploads $\{C, \Omega = \{\sigma_i\}_{1 \le i \le n}, T\}$ to CSS. Finally, F_z saves $H(C)$ and Ω locally for edge duplicate checks. If D_s uploads data in the future, F_z will remove duplicate data based on the already stored block identifiers to ensure that redundant data is not forwarded to CSS. This not only prevents bandwidth usage but also saves cloud storage space costs.

– CSS verifies the consistency between the uploaded data block and the corresponding file identifier by checking $e(T, g) \stackrel{?}{=} e(H(C)^{H_2(AID_z)}, X)$. CSS stores the data only upon successful verification, rejecting unauthorized uploads to prevent non-delegated F_z from impersonation and resource occupation. For file-level deduplication, CSS compares the file tag T with stored tags for files from the same F_z, while block-level deduplication is achieved through uploaded block tags.

5.4 Audit Interaction

F_z generates a random challenge Q for CSS. CSS computes audit proof P based on challenge Q and returns it to F_z for verification. After verifying, F_z publish audit logs to the blockchain.

– Periodically, F_z generates a random challenge $Q = \{i, v_i\}_{i \in I_c}$. I_c is a randomly selected set of c integers from the set of integers $[1, n]$, while $v_i \in \mathbb{Z}_q^*$ is a randomly selected value. Later, F_z will send Q to CSS.
– After receiving the random challenge Q, CSS computes and returns integrity proof $P = \{\mu = \{\mu_j\}_{1 \le j \le \eta}, \rho\}$ as follows:

$$\mu_j = \sum_{i=1}^{c} v_i c_{ij}, \quad \rho = \prod_{i=1}^{c} \sigma_i^{v_i}. \tag{5}$$

– F_z verifies the effectiveness of P through Eq. (6):

$$e(\rho, g) \stackrel{?}{=} e\left(\Lambda \cdot \prod_{j=1}^{\eta} H(\mu_j)^{\varsigma_j} \cdot E, \Gamma\right), \tag{6}$$

where $\Lambda = \prod_{i=1}^{c} H(c_{i1}||\cdots||c_{i\eta})^{v_i}$, $E = \prod_{i=1}^{c} H(a_i)^{v_i}$, and $\Gamma = V_{s,z} \cdot \left(X^{H_2(AID_s)}\right)^{H_1(\omega, V_{s,z})}$. To speed up the calculation process, F_z can precalculate the value of Λ, E, Γ. If the above equation holds, then F_z considers that CSS correctly stores outsourced data and records result $b = 1$; Otherwise, F_z believes that CSS did not honestly store the outsourced files, records $b = 1$, and will immediately notify D_s. Because once the data is damaged, D_s as the data owner needs to know immediately for subsequent processing. Subsequently, F_z creates an audit log $\{T, \{Q, \{\mu, \rho\}, b\}\}$ and uploads it to the blockchain for storage. Assuming that at the end of a task cycle, F_z has completed K audits, then F_z will record the log as $\{T, \{Q^k, \{\mu^k, \rho^k\}, b^k\}\}_{\le k \le K}\}$, where $\{Q^k, \{\mu^k, \rho^k\}, b^k\}$ represents the random challenge, relevant proofs, and audit results of the k-th audit, respectively.

5.5 Reward Incentive

When D_s receives an audit result, it retrieves the auxiliary information from the blockchain to determine the correctness of the received result. If the result is confirmed to be correct and the identity information provided by F_z is verified, the blockchain issues the reward associated with the assigned task to F_z.

- When an audit is completed, D_s can check the results of any of the audits during the assignment. Assuming that D_s wishes to check the k-th audit, D_s checks audit logs $\{Q^k, \{\mu^k, \rho^k\}, b^k\}$ by verifying Eq. (7).

$$e(\rho^k, g) \stackrel{?}{=} e\left(\Lambda \cdot \prod_{j=1}^{\eta} H(\mu_j^k)^{\varsigma_j} \cdot E, \Gamma\right). \tag{7}$$

  If the audit result b^k matches expectations, D_s accepts F_z's result as valid; only when all results are verified does D_s pay F_z, with the option to perform random sampling rather than checking all logs for efficiency.
- After audit verification, F_z submits sk_z to BM for identity validation, where BM checks $g^{sk_z} \stackrel{?}{=} X^{H_2(AID_z)}$ using stored $\{\omega, AID_s, AID_z\}$ and transfers funds from D_s to F_z upon success, ensuring impersonation resistance since attackers cannot forge sk_z and only PKG can generate valid $F_z's$ identities.

6 Analysis of Correctness and Security

In this section, we will provide a correctness analysis and security analysis of the proposed solution.

6.1 Correctness Analysis

(1) *Identity verification*: D_s and F_z accepts the private key and anonymous identity information from PKG after consistency verification through $g^{sk} = g^{xH_2(AID)} = X^{H_2(AID)}$.

(2) *Delegated information verification*: F_z needs to perform delegated information validation before receiving the delegated tasks from D_s, which can be proved by: $g^{sk_{s,z}} = g^{\nu_{s,z}+sk_sH_1(\omega,V_{s,z})} = g^{\nu_{s,z}} \cdot g^{sk_sH_1(\omega,V_{s,z})} = V_{s,z} \cdot \left(g^{sk_s}\right)^{H_1(\omega,V_{s,z})} = V_{s,z} \cdot \left(X^{H_2(AID_s)}\right)^{H_1(\omega,V_{s,z})}$.

(3) *Data file validation*: In order to prevent non-delegated entities from uploading data to the cloud by pretending to be delegated entities and occupying cloud resources, CSS needs to validate the data information, which can be supported to hold: $e(T, g) = e(H(C)^{sk_z}, g) = e(H(C)^{xH_2(AID_z)}, g) = e(H(C)^{H_2(AID_z)}, X)$.

(4) *Audit proof verification*: After issuing a random challenge, F_z needs to validate the audit evidence returned by CSS through Eq. (6) as follows:

$$\begin{aligned} e(\rho, g) &= e\left(\prod_{i=1}^{c} \sigma_i^{v_i}, g\right) \\ &= e\left(\prod_{i=1}^{c} \left(H(c_{i1}||\cdots||c_{i\eta}) \cdot H\left(\sum \varsigma_j c_{ij} + a_i\right)\right)^{v_i sk_{s,z}}, g\right) \\ &= e\left(\prod_{i=1}^{c} H(c_{i1}||\cdots||c_{i\eta})^{v_i} \cdot \prod_{i=1}^{c} H\left(\sum \varsigma_j c_{ij} + a_i\right)^{v_i}, g^{sk_{s,z}}\right) \\ &= e\left(\Lambda \cdot \prod_{i=1}^{c} H\left(\sum \varsigma_j c_{ij} + a_i\right)^{v_i}, V_{s,z} \cdot \left(X^{H_2(AID_s)}\right)^{H_1(\omega, V_{s,z})}\right) \\ &= e\left(\Lambda \cdot \prod_{i=1}^{c} H\left(\sum \varsigma_j c_{ij} + a_i\right)^{v_i}, \Gamma\right) \\ &= e\left(\Lambda \cdot \prod_{j=1}^{\eta} H(\mu_j)^{\varsigma_j} \cdot E, \Gamma\right). \end{aligned} \tag{8}$$

6.2 Security Analysis

(1) *Data privacy security*: During the data upload phase, D_s encrypts the data sectors $\{c_{ij}\}$ through AES.Enc(k_{ij}, m_{ij}) algorithm, which ensures that no malicious entity other than D_s itself is able to recover the plaintext with an unknown encryption key k_{ij}, realizing the privacy protection of the data.

(2) *Forgery Attacks*: To demonstrate the tagging unforgeability of our schemethe, we construct the following security game *Game 1* for the interaction of challenger $\mathcal{C}$ and adversary $\mathcal{A}$:
- *Initialization*: $\mathcal{C}$ generates system parameters for $\mathcal{A}$ with the public key and necessary access rights ($\mathcal{A}_2$ additionally gets the master key);
- *Query*: $\mathcal{A}$ can adaptively make data block tag generation requests.
- *Forgery*: $\mathcal{A}$ outputs a forged tag or false audit proof for a data block;$\mathcal{A}$ outputs a forged tag σ^* for an unqueried block m^*.
- *Verification*: If the forgery passes the verification, $\mathcal{A}$ wins.

Theorem 1. *Our scheme is based on the CDH Assumption that satisfies tag unforgeability.*

Proof. Assume there exists a polynomial-time adversary $\mathcal{A}_1$ that can forge a valid tag with non-negligible probability ϵ. We construct a simulator $\mathcal{B}$ that solves the CDH problem:

- *CDH Instance:* $\mathcal{B}$ receives $(g, g^{\alpha}, g^{\beta})$ and aims to compute $g^{\alpha\beta}$.
- *Simulation:*
  - *Setup:* $\mathcal{B}$ sets the public key $X = g^{\alpha}$ (unknown secret $x = \alpha$).
  - *Hash Queries:* For $H(c_{i1}||\cdots||c_{i\eta})$, $\mathcal{B}$ programs the random oracle to return g^{r_i} where $r_i \leftarrow \mathbb{Z}_q^*$. For $H\left(\sum_{j=1}^{\eta} \varsigma_j c_{ij} + a_i\right)$, the random oracle returns g^{s_i} where $s_i \leftarrow \mathbb{Z}_q^*$.
  - *Tag Generation:* For query (c_i, a_i), $\mathcal{B}$ computes $\sigma_i = (g^{r_i} \cdot g^{s_i})^{\alpha} = (g^{\alpha})^{r_i + s_i}$.
- *Forgery:* $\mathcal{A}_1$ outputs a forged tag σ^* for new block c^* where:

$$\sigma^* = \left(H(c_1^*||\cdots||c_{\eta}^*) \cdot H\left(\sum_{j=1}^{\eta} \varsigma_j c_{ij}^* + a^*\right)\right)^{\alpha} = \left(g^{r^*} \cdot g^{s^*}\right)^{\alpha} = (g^{\alpha})^{r^* + s^*}$$

.

- $\mathcal{B}$ extracts solution to *CDH* as: $g^{\alpha\beta} = \left(\sigma^* \cdot (g^{\alpha})^{-(r^* + s^*)}\right)^{1/\gamma}$, where $\gamma = H_1(\omega, V_{s,z})$. If $\mathcal{A}_1$ succeeds with probability ϵ, $\mathcal{B}$ solves the CDH instance with probability $\epsilon' \geq \epsilon/(q_H \cdot q_T)$, where q_H and q_T are hash and tag queries, contradicting the CDH assumption.

Theorem 2. *Our scheme is based on the DL Assumption that satisfies tag unforgeability.*

Proof. If there exists internal adversary $\mathcal{A}_2$ that breaks *Game 1* with non-negligible probability ϵ, then algorithm $\mathcal{B}$ can be constructed to solve the *DL Assumption* with probability $\epsilon' \approx \epsilon$.

- *DL Instance:* $\mathcal{B}$ receives (g, g^{α}) and aims to find α.
- *Simulation:*
  - $\mathcal{B}$ gives the master key α to $\mathcal{A}_2$.
  - Programs $H(c_{i1}||\cdots||c_{i\eta})$ to return random $h_i \in \mathbb{Z}_q^*$.
- *Forgery:* $\mathcal{A}_2$ outputs $\sigma^* = (h^* \cdot H(\cdot)^*)^{\alpha}$.

$\mathcal{B}$ computes $\alpha = \frac{\log_g \sigma^*}{\log_g h^* + \log_g H(\cdot)}$. If $\mathcal{A}_2$ succeeds with probability ϵ, $\mathcal{B}$ solves *DL*, contradicting DL assumption.

Under the CDH and DL assumption, no polynomial-time adversary can forge valid tags in the scheme, thus proving that the scheme satisfies tag unforgeability.

7 Experimental Evaluation

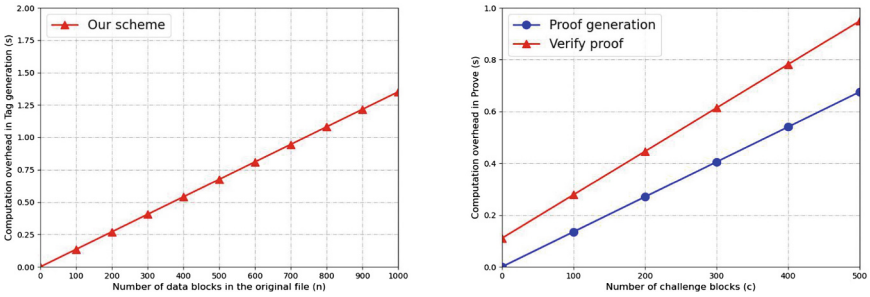
In this section, we evaluate the performance of the proposed delegatable auditing scheme with edge deduplication and privacy protection. The experiments focus on computational overhead and communication overhead. All experiments are conducted on Ubuntu 24.04.1 LTS with 2 CPUs, 4 GB RAM, and 20 GB storage. Cryptographic operations are implemented using Python with the PBC and GMP libraries [22].

7.1 Computational Overhead

The computational operations in our scheme primarily consist of three phases: *tag generation*, *audit proof generation*, and *audit verification*. Let n denote the number of data blocks, η the sector count per block, and c the number of challenged blocks. The notations Exp, Mul, and $Pair$ represent one exponentiation, one multiplication, and one pairing operation in $\mathbb{G}_1$ or $\mathbb{G}_2$, respectively.

- *Tag Generation:* Each fog node F_z computes block tags σ_i for n blocks, requiring n exponentiations $nExp$ and $n\eta$ multiplications $n(\eta+1)Mul$ for MLE encryption. The total cost is $nExp + n(\eta+1)Mul$.
- *Audit Proof Generation:* CSS costs $cExp$ and $(c\eta + c - 1)Mul$ for proof P, totaling $cExp + n(\eta+1)Mul$.
- *Audit Verification:* F_z verifies the proof using Eq. (6), requiring $2Pair$ for pairing operations, $(2c+\eta+2)Exp$ for exponentiations, and $(2c+\eta)Mul$ for multiplications. The total cost is $(2c+\eta+2)Exp + (2c+\eta)Mul + 2Pair$.

Figure 2 shows the computational overhead for different block counts ($n = 100$ to 1000) with $\eta = 128$ and challenged block count ($c = 100$ to 500). The results demonstrate that tag generation scales linearly with n, while audit phases remain efficient due to precomputed values like Γ.



(a) Upload (b) Auditing

Fig. 2. Computational overhead in data upload and audit phases.

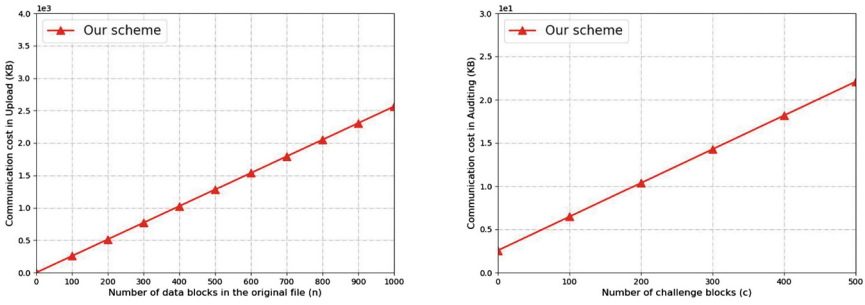
7.2 Communication Overhead

The communication costs are dominated by:

- *Data Upload:* Each block tag σ_i is one element in $\mathbb{G}_1$, and the ciphertext C requires $n\eta|\mathbb{Z}_q^*|$. In addition, the file tag T is also an element in $\mathbb{G}_1$. The total upload cost is $(n+1)|\mathbb{G}_1| + n\eta|\mathbb{Z}_q^*|$.

– *Audit Challenge/Response:* The challenge Q contains c block indices and random values ($2c|\mathbb{Z}_q^*|$), while the proof $P = \{\mu, \rho\}$ consists of η field elements and one group element ($\eta|\mathbb{Z}_q^*|+|\mathbb{G}_1|$). The total audit cost is $|\mathbb{G}_1|+(2c+\eta)|\mathbb{Z}_q^*|$.

Figure 3 compares the communication overhead for varying n and η. When $\eta = 128$, the upload cost is lower, verifying the efficiency of our block splitting technique.



(a) Upload (b) Auditing

Fig. 3. Communication overhead in data upload and audit phases.

The experimental results confirm the practicality of our scheme for IoT environments. Edge deduplication reduces cloud storage and bandwidth usage compared to cloud-only approaches, while the audit phase completes within 0.88s for 460 blocks, meeting real-time requirements.

8 Conclusion

In this paper, we proposed a delegatable auditing scheme with edge deduplication and privacy protection for IoT environments. Our scheme enables lightweight data integrity verification while addressing bandwidth and storage inefficiencies through edge-assisted deduplication. We presented the detailed construction, rigorous correctness proofs, and comprehensive security analysis under the CDH and DL assumptions. Experimental evaluations demonstrate that the scheme achieves practical efficiency, reducing cloud storage overhead. The results confirm that our approach outperforms existing solutions in balancing security, privacy, and performance for resource-constrained IoT systems.

Acknowledgments. This work is supported by National Natural Science Foundation of China (62302330, 62402397), the Natural Science Foundation of Jiangsu Province (BK20230727), Suqian Sci&Tech Program of Jiangsu Province (K202229, M202305), Natural Science Research Project Funding for Higher Education Institutions of Jiangsu Province (23KJD520013), Youth Science and Technology Talent Support Project of Jiangsu Province (JSTJ-2024-662).

References

1. Zhou, L., Fu, A., Yu, S., Su, M., Kuang, B.: Data integrity verification of the outsourced big data in the cloud environment: a survey. J. Netw. Comput. Appl. **122**, 1–15 (2018)
2. Shi, W., Cao, J., Zhang, Q., Li, Y., Xu, L.: Edge computing: vision and challenges. IEEE Internet Things J. **3**(5), 637–646 (2016)
3. Hazra, A., Rana, P., Adhikari, M., Amgoth, T.: Fog computing for next-generation internet of things: fundamental, state-of-the-art and research challenges. Comput. Sci. Rev. **48**, 100549 (2023)
4. Yeh, J.H., Arifin, M.M., Shen, N., Karki, U., Xie, Y., Nanjundarao, A.: Integrity coded databases - protecting data integrity for outsourced databases. Comput. Secur. **136**, 103569 (2024)
5. Tian, H., Nan, F., Chang, C.C., Huang, Y., Lu, J., Du, Y.: Privacy-preserving public auditing for secure data storage in fog-to-cloud computing. J. Netw. Comput. Appl. **127**, 59–69 (2019)
6. Ateniese, G., et al.: Provable data possession at untrusted stores. In: Proceedings of the 14th ACM Conference on Computer and Communications Security, CCS 2007, pp. 598–609. Association for Computing Machinery, New York (2007)
7. Hasan, M.Z., Hussain, M.Z., Mubarak, Z., Siddiqui, A.A., Qureshi, A.M., Ismail, I.: Data security and integrity in cloud computing. In: 2023 International Conference for Advancement in Technology (ICONAT), pp. 1–5 (2023)
8. Peng, L., Yan, Z., Liang, X., Yu, X.: Secdedup: secure data deduplication with dynamic auditing in the cloud. Inf. Sci. **644**, 119279 (2023)
9. Tian, G., et al.: Blockchain-based secure deduplication and shared auditing in decentralized storage. IEEE Trans. Dependable Secure Comput. **19**(6), 3941–3954 (2022)
10. Song, M., Hua, Z., Zheng, Y., Xiang, T., Jia, X.: Enabling transparent deduplication and auditing for encrypted data in cloud. IEEE Trans. Dependable Secure Comput. **21**(4), 3545–3561 (2024)
11. Hahn, C., Kwon, H., Kim, D., Hur, J.: Enabling fast public auditing and data dynamics in cloud services. IEEE Trans. Serv. Comput. **15**(4), 2047–2059 (2022)
12. Zhou, L., Fu, A., Yang, G., Wang, H., Zhang, Y.: Efficient certificateless multi-copy integrity auditing scheme supporting data dynamics. IEEE Trans. Dependable Secure Comput. **19**(2), 1118–1132 (2022)
13. Li, T., Chu, J., Hu, L.: Cia: A collaborative integrity auditing scheme for cloud data with multi-replica on multi-cloud storage providers. IEEE Trans. Parallel Distrib. Syst. **34**(1), 154–162 (2023)
14. Gai, C., Shen, W., Yang, M., Yu, J.: PPADT: privacy-preserving identity-based public auditing with efficient data transfer for cloud-based IoT data. IEEE Internet Things J. **10**(22), 20065–20079 (2023)
15. Zhang, Q., et al.: Efficient blockchain-based data integrity auditing for multi-copy in decentralized storage. IEEE Trans. Parallel Distrib. Syst. **34**(12), 3162–3173 (2023)
16. Tian, Y., Tan, H., Shen, J., Pandi, V., Gupta, B.B., Arya, V.: Efficient identity-based multi-copy data sharing auditing scheme with decentralized trust management. Inf. Sci. **644**, 119255 (2023)
17. Yang, Y., Chen, Y., Xiong, P., Chen, F., Chen, J.: Decentralized self-auditing multiple cloud storage in compressed provable data possession. IEEE Trans. Dependable Secure Comput. (2025). https://doi.org/10.1109/TDSC.2025.3542068

18. Xu, Y., Zhang, C., Wang, G., Qin, Z., Zeng, Q.: A blockchain-enabled deduplicatable data auditing mechanism for network storage services. IEEE Trans. Emerg. Top. Comput. **9**(3), 1421–1432 (2021)
19. Gu, K., Wang, Y., Qiu, J., Li, X., Zhang, J.: Blockchain-based data deduplication and distributed audit for shared data in cloud-fog computing-based vanets. IEEE Trans. Netw. Serv. Manage. **21**(5), 5548–5565 (2024)
20. Liu, B., Zhang, X., Yang, X., Zhang, Y., Xue, J., Zhou, R.: Blockchain-assisted fine-grained deduplication and integrity auditing for outsourced large-scale data in cloud storage. IEEE Internet Things J. (2025). https://doi.org/10.1109/JIOT.2025.3548681
21. Zhang, Q., Qian, S., Cui, J., Zhong, H., Wang, F., He, D.: Blockchain-based privacy-preserving deduplication and integrity auditing in cloud storage. IEEE Trans. Comput. **74**(5), 1717–1729 (2025)
22. The pairing-based cryptography library (PBC). http://crypto.stanford.edu/pbc/howto.html